Laporan Praktikum Algoritma & Pemrograman

Semester Genap 2025/2026

SAYA MENYATAKAN BAHWA LAPORAN PRAKTIKUM INI SAYA BUAT DENGAN USAHA SENDIRI TANPA MENGGUNAKAN BANTUAN ORANG LAIN. SEMUA MATERI YANG SAYA AMBIL DARI SUMBER LAIN SUDAH SAYA CANTUMKAN SUMBERNYA DAN TELAH SAYA TULIS ULANG DENGAN BAHASA SAYA SENDIRI.

SAYA SANGGUP MENERIMA SANKSI JIKA MELAKUKAN KEGIATAN PLAGIASI, TERMASUK SANKSI TIDAK LULUS MATA KULIAH INI.

NIM	**71200556**
Nama Lengkap	**Frans Sailapra**
Minggu ke / Materi	**04 / Struktur Kontrol Percabangan**

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS KRISTEN DUTA WACANA
YOGYAKARTA
2026

BAGIAN 1: MATERI MINGGU INI (40%)

Pada bagian ini, tuliskan kembali semua materi yang telah anda pelajari minggu ini. Sesuaikan penjelasan anda dengan urutan materi yang telah diberikan di saat praktikum. Penjelasan anda harus dilengkapi dengan contoh, gambar/ilustrasi, contoh program (source code) dan outputnya. Idealnya sekitar 5-6 halaman.

MATERI 1 : Boolean Expression dan Logical Operator

A. Boolean Expression

Boolean expression adalah suatu ekspresi yang menghasilkan nilai logika, yaitu hanya memiliki dua kemungkinan hasil: True atau False. Biasanya boolean expression menggunakan operator perbandingan seperti:

- > (lebih besar dari)
- < (lebih kecil dari)
- >= (lebih besar atau sama dengan)
- <= (lebih kecil atau sama dengan)
- == (sama dengan)
- != (tidak sama dengan)

Contoh kasus:
Misalnya terdapat promo voucher diskon 30% dengan syarat minimum pembelian Rp100.000. Artinya, pelanggan hanya bisa mendapatkan diskon jika total belanja minimal Rp100.000. Dalam Python, syarat tersebut dapat dituliskan sebagai:
<u>pembelian >= 100000</u>

Ekspresi ini disebut boolean expression karena hasil evaluasinya hanya akan bernilai True atau False, tergantung pada nilai variabel pembelian.

B. Logical Operator

Selain operator perbandingan, dalam Python juga terdapat *logical operator* yang digunakan untuk menggabungkan dua atau lebih kondisi. Operator logika yang umum digunakan adalah:

- and, Menghasilkan True jika kedua kondisi bernilai True.
  Contoh : pembelian >= 100000 and member == True
  Artinya, diskon hanya berlaku jika pembelian memenuhi minimum dan pelanggan adalah member.
- or, Menghasilkan True jika salah satu kondisi bernilai True.
  Contoh : pembelian >= 100000 or memiliki_voucher == True
  Artinya, diskon bisa berlaku jika salah satu syarat terpenuhi.

- Not, Digunakan untuk membalik nilai boolean. Jika kondisi True maka menjadi False, dan sebaliknya.

MATERI 2 : Bentuk-bentuk Percabangan

Dalam Python, percabangan digunakan untuk menentukan tindakan apa yang akan dijalankan berdasarkan suatu kondisi tertentu. Percabangan ini sangat berkaitan dengan boolean expression yang sudah dibahas sebelumnya, karena setiap percabangan selalu bergantung pada nilai True atau False dari suatu kondisi.

Secara umum, bentuk percabangan dalam Python dibagi menjadi tiga, yaitu: **conditional**, **alternative conditional**, dan **chained conditional**.

1. Conditional (If Tunggal)
   Bentuk conditional adalah percabangan paling sederhana. Pada bentuk ini, perintah hanya akan dijalankan jika kondisi bernilai True. Jika kondisi False, maka program akan melewati bagian tersebut tanpa melakukan apa-apa.
   Struktur umumnya:
   if <kondisi>:
   <lakukan ini>
   <lakukan ini>

   Contoh:
   Jika nilai akhir lebih dari 70, maka siswa mendapatkan sertifikat kelulusan.
   ```
   if nilai_akhir > 70:
   print("Anda lulus dan mendapatkan sertifikat kelulusan!")
   ```
   Artinya, jika nilai_akhir > 70 bernilai True, maka perintah print akan dijalankan. Jika tidak, program tidak menampilkan apa-apa.

2. Alternative Conditional (If-Else)
   Bentuk ini digunakan jika terdapat dua kemungkinan tindakan. Jadi, jika kondisi pertama tidak terpenuhi, maka program akan menjalankan perintah alternatif.
   Struktur umum:
   if <kondisi>:
   <lakukan ini>
   else:
   <lakukan itu>

   Contoh:
   Jika nilai akhir lebih dari 60 maka Lulus, jika tidak maka Tidak Lulus.
   ```
   if nilai_akhir > 60:
   print("Lulus")
   ```

```
else:
   print("Tidak Lulus")
```

ini hanya ada dua kemungkinan hasil, yaitu “Lulus” atau “Tidak Lulus”. Program tidak akan menghasilkan pilihan lain selain dua kemungkinan tersebut.

3. Chained Conditional (If-Elif-Else)
   Chained conditional digunakan jika terdapat lebih dari dua kemungkinan kondisi. Dalam hal ini digunakan elif (else if) untuk menambahkan kondisi tambahan.
   Struktur umum:

```
if <kondisi 1>:
   <aksi A>
elif <kondisi 2>:
   <aksi B>
elif <kondisi 3>:
   <aksi C>
else:
   <aksi lainnya>
```

   Contoh:
   Sebuah toko memberikan diskon berdasarkan jumlah pembelian:
   Rp1.000.000 → diskon 30%
   Rp500.000 – Rp1.000.000 → diskon 20%
   Rp100.000 – Rp500.000 → diskon 15%
   < Rp100.000 → tidak ada diskon
   Implementasinya :

```
if pembelian > 1000000:
   diskon = 0.3  # diskon 30%
elif pembelian > 500000 and pembelian <= 1000000:
   diskon = 0.2  # diskon 20%
elif pembelian >= 100000 and pembelian <= 500000:
   diskon = 0.15  # diskon 15%
else:
   diskon = 0  # tidak ada diskon
```

4. Ternary Operator
   Selain bentuk percabangan biasa, Python juga menyediakan bentuk singkat yang disebut ternary operator. Bentuk ini biasanya digunakan untuk percabangan sederhana dengan dua kemungkinan.

Contoh :

```
pembelian = int(input("Jumlah pembelian: "))
diskon = 0.1 if pembelian > 100000 else 0
```

Bentuk tersebut sebenarnya sama dengan:

```
pembelian = int(input("Jumlah pembelian: "))
if pembelian > 100000:
  diskon = 0.1
else:
  diskon = 0
```

Perbedaannya hanya pada cara penulisan. Ternary operator lebih ringkas, tetapi biasanya digunakan untuk kondisi yang sederhana agar tetap mudah dibaca.

MATERI 3 : Penanganan Kesalahan Input Menggunakan Exception Handling

Bahian ini membahas bagaimana menangani kesalahan input (input error) dari pengguna menggunakan mekanisme **exception handling** pada Python. Studi kasus yang digunakan adalah program untuk menentukan kategori usia berdasarkan input yang dimasukkan oleh pengguna.

Program meminta pengguna memasukkan usia, kemudian menentukan kategori sebagai berikut:

- Balita: 0–5 tahun
- Kanak-kanak: 6–11 tahun
- Remaja: 12–25 tahun
- Dewasa: 26–45 tahun
- Lansia: > 45 tahun

Potongan kode utama:

```
usia = int(input("Masukkan usia anda: "))
```

Kemudian dilakukan percabangan if-elif untuk menentukan kategori usia.

Pada Gambar 4.2 terlihat beberapa kali eksekusi program dengan input yang berbeda, misalnya:

- Input: 33 → Output: Dewasa
- Input: 7 → Output: Kanak-kanak
- Input: 68 → Output: Lansia
- Input: 18 → Output: Remaja
- Input: 2 → Output: Balita

```
yuanlukito@linuxstation: ~/Documents
yuanlukito@linuxstation:~/Documents$ python3 kategoriusia.py
Masukkan usia anda: 33
Dewasa
yuanlukito@linuxstation:~/Documents$ python3 kategoriusia.py
Masukkan usia anda: 7
Kanak-kanak
yuanlukito@linuxstation:~/Documents$ python3 kategoriusia.py
Masukkan usia anda: 68
Lansia
yuanlukito@linuxstation:~/Documents$ python3 kategoriusia.py
Masukkan usia anda: 18
Remaja
yuanlukito@linuxstation:~/Documents$ python3 kategoriusia.py
Masukkan usia anda: 2
Balita
yuanlukito@linuxstation:~/Documents$
```

Gambar 4.2: Hasil program kategori usia.

Hal ini menunjukkan bahwa:

- Program berjalan dengan baik jika input berupa angka.
- Percabangan kondisi bekerja sesuai logika yang telah ditentukan.

Masalah muncul ketika pengguna memasukkan input yang tidak sesuai, misalnya:
Masukkan usia anda: dua puluh
Karena fungsi input() menghasilkan string, sedangkan variabel usia harus berupa **integer**, maka dilakukan konversi menggunakan int().

Jika string tersebut tidak dapat dikonversi menjadi angka, Python menghasilkan error:
ValueError: invalid literal for int() with base 10

```
yuanlukito@linuxstation: ~/Documents
yuanlukito@linuxstation:~/Documents$ python3 kategoriusia.py
Masukkan usia anda: dua puluh
Traceback (most recent call last):
  File "kategoriusia.py", line 1, in <module>
    usia = int(input("Masukkan usia anda: "))
ValueError: invalid literal for int() with base 10: 'dua puluh'
yuanlukito@linuxstation:~/Documents$
```

Gambar 4.3: Jika input tidak sesuai yang diharapkan, program akan berhenti.

Seperti terlihat pada Gambar 4.3:

- Program langsung berhenti (crash)
- Tidak menampilkan pesan yang ramah pengguna
- Menghasilkan traceback error

Ini menunjukkan bahwa program belum memiliki mekanisme penanganan kesalahan.

Solusi Menggunakan try–except (Exception Handling)
Untuk mengatasi masalah tersebut, digunakan struktur:

```
try:
  usia = int(inputuser)
  ...
except:
```

```
print("Anda salah memasukkan input usia")
```

Struktur ini bekerja sebagai berikut:

- try → Menjalankan kode yang berpotensi menghasilkan error.
- except → Menangani kesalahan jika terjadi error di dalam blok try.

Gambar 4.4: Hasil program kategori usia.

Pada Gambar 4.4 ditunjukkan bahwa ketika pengguna memasukkan input: dua puluh Program tidak lagi menampilkan error teknis, melainkan menampilkan pesan: Anda salah memasukkan input usia

Artinya:

- Program tidak berhenti secara paksa.
- Kesalahan ditangani dengan baik.
- Pengalaman pengguna menjadi lebih baik (user-friendly).

BAGIAN 2: LATIHAN MANDIRI (60%)

Pada bagian ini anda menuliskan jawaban dari soal-soal Latihan Mandiri yang ada di modul praktikum. Jawaban anda harus disertai dengan source code, penjelasan dan screenshot output.

SOAL 1 : Implementasikan penanganan kesalahan input pengguna dari program-program pada Contoh 3.1, 3.2 dan 3.3.

soal01.py

```python
# latihan 3.1
while True:
    inputuser = input("Masukkan sebuah bilangan: ")

    try:
        angka = int(inputuser)

        if angka <= 1:
            print("Bukan bilangan prima")
        else:
            prima = True
            for i in range(2, angka):
                if angka % i == 0:
                    prima = False
                    break

            if prima:
                print("Bilangan Prima")
            else:
                print("Bukan Bilangan Prima")

        break

    except ValueError:
        print("Input harus berupa angka bulat!")
```

soal01.py

```python
# latihan 3.2
try:
    a = float(input("Masukkan nilai a: "))
    b = float(input("Masukkan nilai b: "))
    c = float(input("Masukkan nilai c: "))

    D = b**2 - 4*a*c

    if D > 0:
        print("Akar Real dan Berlainan")
    elif D == 0:
        print("Akar Real dan Kembar")
    else:
        print("Akar Imajiner / Tidak Real")

except ValueError:
    print("Input harus berupa angka!")
```

soal01.py

```python
# latihan 3.3
while True:
    inputuser = input("Masukkan bilangan untuk dihitung faktorial: ")

    try:
        n = int(inputuser)

        if n < 0:
            print("Bilangan tidak boleh negatif!")
            continue

        faktorial = 1
        for i in range(1, n + 1):
            faktorial *= i

        print("Nilai faktorial =", faktorial)
        break

    except ValueError:
        print("Input harus berupa angka bulat!")
```

SOAL 2 : Implementasikan percabangan pada Contoh 3.2 (Positif-Negatif) menggunakan ternary operator.

soal02.py

```python
a = float(input("Masukkan nilai a: "))
b = float(input("Masukkan nilai b: "))
c = float(input("Masukkan nilai c: "))

D = b**2 - 4*a*c

if D > 0:
    print("Akar Real dan Berlainan")
elif D == 0:
    print("Akar Real dan Kembar")
else:
    print("Akar Imajiner / Tidak Real")
```

SOAL 3 : Buatlah sebuah program yang dapat menampilkan jumlah hari dalam suatu bulan di tahun 2020. Program meminta pengguna memasukkan nomor bulan (1-12), kemudian program akan menampilkan jumlah hari pada bulan tersebut.

soal03.py

```python
while True:
    inputuser = input("Masukkan bulan (1-12): ")

    try:
        bulan = int(inputuser)

        if bulan < 1 or bulan > 12:
            print("Bulan yang diinputkan tidak valid!")
            continue

        if bulan == 2:
            jumlah_hari = 29  # Tahun 2020 (kabisat)
        elif bulan in [4, 6, 9, 11]:
            jumlah_hari = 30
        else:
            jumlah_hari = 31

        print("Jumlah hari:", jumlah_hari)
        break

    except ValueError:
        print("Input harus berupa angka antara 1 sampai 12!")
```

SOAL 4 : Sebuah program meminta pengguna memasukkan ketiga panjang sisi suatu segitiga (berarti pengguna memasukkan tiga bilangan). Jika ketiga sisi segitiga tersebut semuanya sama, tampilkan pesan: "3 sisi sama". Jika hanya ada dua sisi yang sama panjang, tampilkan pesan "2 sisi sama". Jika tidak ada yang sama maka tampilkan pesan: "Tidak ada yang sama".

soal04.py

```python
while True:
    try:
        sisi1 = float(input("Masukkan sisi 1: "))
        sisi2 = float(input("Masukkan sisi 2: "))
        sisi3 = float(input("Masukkan sisi 3: "))

        if sisi1 <= 0 or sisi2 <= 0 or sisi3 <= 0:
            print("Panjang sisi harus lebih dari 0!")
            continue

        if sisi1 == sisi2 == sisi3:
            print("3 sisi sama")
        elif sisi1 == sisi2 or sisi1 == sisi3 or sisi2 == sisi3:
            print("2 sisi sama")
        else:
            print("Tidak ada yang sama")

        break

    except ValueError:
        print("Input tidak valid! Masukkan angka yang benar.")
```

Pada program di atas blok try digunakan untuk menghindari erroe ketika input, ada validasi nilai positif karena Panjang sisi segitiga tidak boleh nol atau negative.

Github : https://github.com/71200556/71200556_frans